

The Stanford Research Institute Problem Solver, known by its acronym STRIPS, is an automated planner developed by Richard Fikes and Nils Nilsson in 1971 at SRI International. The same name was later used to refer to the formal language of the inputs to this planner. This language is the base for most of the languages for expressing automated planning problem instances in use today; such languages are commonly known as action languages. This article only describes the language, not the planner.

Definition

A STRIPS instance is composed of:

An initial state;

The specification of the goal states – situations that the planner is trying to reach;

A set of actions. For each action, the following are included:

preconditions (what must be established before the action is performed);

postconditions (what is established after the action is performed).

Mathematically, a STRIPS instance is a quadruple

$\langle$
P
,
O
,
I
,

G

}

$$\{\displaystyle \langle P,O,I,G\rangle \}$$

, in which each component has the following meaning:

P

$$\{\displaystyle P\}$$

is a set of conditions (i.e., propositional variables);

O

$$\{\displaystyle O\}$$

is a set of operators (i.e., actions); each operator is itself a quadruple

$\langle$
 α
,
 β
,
 γ
,
 δ
 $\rangle$

$\{\displaystyle \langle \alpha ,\beta ,\gamma ,\delta \rangle \}$

, each element being a set of conditions. These four sets specify, in order, which conditions must be true for the action to be executable, which ones must be false, which ones are made true by the action and which ones are made false;

I

$$I$$

is the initial state, given as the set of conditions that are initially true (all others are assumed false);

$$G$$

$$G$$

is the specification of the goal state; this is given as a pair

$$\langle N, M \rangle$$

$$\langle N, M \rangle$$

, which specify which conditions are true and false, respectively, in order for a state to be considered a goal state.

A plan for such a planning instance is a sequence of operators that can be executed from the initial state and that leads to a goal state.

Formally, a state is a set of conditions: a state is represented by the set of conditions that are true in it. Transitions between states are modeled by a transition function, which is a function mapping states into new states that result from the execution of actions. Since states are represented by sets of conditions, the transition function relative to the STRIPS instance

$\langle$
 P
 $,$
 O
 $,$
 I
 $,$
 G
 $\rangle$

$\{\displaystyle \langle P,O,I,G \rangle\}$

is a function

Succ

:

2

P

x

O

→

2

P

,

$$\{\operatorname{succ} : 2^P \times O \rightarrow 2^P, \}$$

where

$$2$$

$$P$$

$$\{ \displaystyle 2^{\{P\}} \}$$

is the set of all subsets of

$$P$$

$$\{ \displaystyle P \}$$

, and is therefore the set of all possible states.

The transition function

succ

$$\{\text{succ}\}$$

for a state

C

$\subseteq$

P

$$C \subseteq P$$

, can be defined as follows, using the simplifying assumption that actions can always be executed but have no effect if their preconditions are not met:

The function

SUCC

$$\operatorname{succ}$$

can be extended to sequences of actions by the following recursive equations:

SUCC

$$\operatorname{succ}(C,[\])=C$$

$$\operatorname{succ}(C,[\])=C$$

succ

(

C

,

[

a

1

,

a

2

,

...

,

a

n

]

)

=

SUCC

(

SUCC

(

C

,

a

1

)

,

[

a

2

,

...

,

a

n

]

)

$$\{\text{\operatorname{succ}}(C,[a_{\{1\}},a_{\{2\}},\ldots,a_{\{n\}}])=\text{\operatorname{succ}}(\text{\operatorname{succ}}(C,a_{\{1\}}),[a_{\{2\}},\ldots,a_{\{n\}}])\}$$

A plan for a STRIPS instance is a sequence of actions such that the state that results from executing the actions in order from the initial state satisfies the goal conditions. Formally,

[

a

1

,

a

2

,

...

,

a

n

]

$$\{a_1,a_2,\ldots ,a_n\}$$

is a plan for

G

=

<

N

,

M

>

$$G=\langle N,M\rangle$$

if

F

=

succ

(

I

,

[

a

1

,

a

2

,

...

,

a

n

]
)

{\displaystyle F=\operatorname {succ} (I,[a_{1},a_{2},\ldots ,a_{n}])}

satisfies the following two conditions:

N
⊆
F

{\displaystyle N\subseteq F}

M
∩
F
=

∅

$$\{ \displaystyle M \cap F = \varnothing \}$$

Extensions

The above language is actually the propositional version of STRIPS; in practice, conditions are often about objects: for example, that the position of a robot can be modeled by a predicate

A

t

$$\{ \displaystyle A_t \}$$

, and

A

t

(

r

o

o
m
1
)

$$\text{At}(\text{room1})$$

means that the robot is in Room1. In this case, actions can have free variables, which are implicitly existentially quantified. In other words, an action represents all possible propositional actions that can be obtained by replacing each free variable with a value.

The initial state is considered fully known in the language described above: conditions that are not in

I

$$I$$

are all assumed false. This is often a limiting assumption, as there are natural examples of planning problems in which the initial state is not fully known. Extensions of STRIPS have been developed to deal with partially known initial states.

A sample STRIPS problem

A monkey is at location A in a lab. There is a box in location C. The monkey wants the bananas that are hanging from the ceiling in location B, but it needs to move the box and climb onto it in order to reach them.

Initial state: At(A), Level(low), BoxAt(C), BananasAt(B)

Goal state: Have(bananas)

Actions:

// move from X to Y

Move(X, Y)

Preconditions: At(X), Level(low)

Postconditions: not At(X), At(Y)

// climb up on the box

ClimbUp(Location)

Preconditions: At(Location), BoxAt(Location), Level(low)

Postconditions: Level(high), not Level(low)

// climb down from the box

ClimbDown(Location)

Preconditions: At(Location), BoxAt(Location), Level(high)

Postconditions: Level(low), not Level(high)

// move monkey and box from X to Y

MoveBox(X, Y)

Preconditions: At(X), BoxAt(X), Level(low)

Postconditions: BoxAt(Y), not BoxAt(X), At(Y), not At(X)

// take the bananas

TakeBananas(Location)

Preconditions: At(Location), BananasAt(Location), Level(high)

Postconditions: Have(bananas)

Complexity

Deciding whether any plan exists for a propositional STRIPS instance is PSPACE-complete. Various restrictions can be enforced in order to decide if a plan exists in polynomial time or at least make it an NP-complete problem.

Macro operator

In the monkey and banana problem, the robot monkey has to execute a sequence of actions to reach the banana at the ceiling. A single action provides a small change in the game. To simplify the planning process, it make sense to invent an abstract action, which isn't available in the normal rule description. The super-action consists of low level actions and can reach high-level goals. The advantage is that the computational complexity is lower, and longer tasks can be planned by the solver.

Identifying new macro operators for a domain can be realized with genetic programming. The idea is, not to plan the domain itself, but in the pre-step, a heuristics is created that allows the domain to be solved much faster. In the context of reinforcement learning, a macro-operator is called an option. Similar to the definition within AI planning, the idea is, to provide a temporal abstraction (span over a longer period) and to modify the game state directly on a higher layer.

See also

Action description language (ADL)

Automated planning

Hierarchical task network

Planning Domain Definition Language (PDDL)

Sussman anomaly

References

Further reading

C. Bäckström and B. Nebel (1995). Complexity results for SAS+ planning. *Computational Intelligence*, 11:625-656.

T. Bylander (1991). Complexity results for planning. In *Proceedings of the Twelfth International Joint Conference on Artificial Intelligence (IJCAI'91)*, pages 274-279.

Russell, Stuart J.; Norvig, Peter (2003), *Artificial Intelligence: A Modern Approach* (2nd ed.), Upper Saddle River, New Jersey: Prentice Hall, ISBN 0-13-790395-2